

UPDATE

MCSL Release version 1.0.380p

PluginHive MCSL · Shopify / WooCommerce

Support Guide - Story Card Package

Generated May 2026 · PluginHive QA Team

Support Guide - Story Card Package

Generated on May 28, 2026

Included Story Cards

Story / card	Platform	Carrier scope	Toggle / prerequisite signal
ZI-560	Shopify	FedEx REST customs / label request	No merchant-facing toggle detected
Internal	Shopify	Order Grid filter search	No toggle detected
ZI-555	Shopify	FedEx REST address handling for HK / Taiwan	No merchant-facing toggle detected
ZI-520	Shopify	Amazon Shipping India	Amazon Shipping India account and VA-capable flow required
ZI-557	WooCommerce	WooCommerce Shipping Services checkout rates	Shipping zone postcode configuration must exist
Internal	Shopify	FedEx REST rates for GU and VI	No merchant-facing toggle detected
Internal	Shopify	FedEx app and MCSL mock rates	Store-based internal QA toggle required

How Support Should Use This Package

- › Use each card section as the support and demo guide for that change.
- › Confirm store, platform, carrier account, release/build, and toggle state before promising behavior.
- › This package was prepared from the current MCSL repo patterns and the card titles provided for these items. If a live Trello comment, QA note, or request log shows a narrower scope, prefer the live evidence.
- › For request-log checks, capture screenshots of the exact request, response, or rate-log area rather than paraphrasing what you saw.
- › The mock-rates section is for internal QA/support handling only and should not be positioned as a merchant feature unless product or QA explicitly says so.

CARD 1/7

ZI-560 - Round customs value to 2 decimals before sending to FedEx

Feature Summary

This change is intended to stop FedEx label rejections caused by float-artifact customs values such as long decimal expansions coming out of product, discount, or customs calculations. FedEx should receive customs values rounded to two decimal places before the shipment request is sent, so international label generation does not fail on otherwise valid orders.

Support should treat this as a request-payload correctness fix. The merchant should not need to change how they enter product values. The visible outcome is that the same shipment now rates or labels successfully, and the FedEx request log shows clean two-decimal customs amounts instead of long floating-point values.

Toggles & Prerequisites

Check	What to confirm
Release	Confirm the store is on the build containing ZI-560
Carrier account	FedEx REST is the active carrier for the tested flow
Shipment scope	Use an international or customs-required shipment where FedEx receives commodity customs values
Product data	Product customs value or derived declared value contains decimals that previously could expand unexpectedly
Toggle	No merchant-facing toggle detected locally

Step-by-Step Support Walkthrough

1. Open Shopify Admin > Apps > PH Multi Carrier Shipping Label App and confirm the affected store is using FedEx REST for the target shipment.
2. Use or create an order whose product customs value, discounted line value, or declared amount is likely to produce a decimal-heavy result.
3. Open the order in ORDERS and run the normal FedEx REST label flow.
4. Confirm the label request no longer fails with a FedEx validation or bad-request style rejection tied to customs amount formatting.
5. Open the request log for the same run and inspect the commodity customs-value field in the outbound FedEx shipment payload.
6. Request/log fields to verify: `customsClearanceDetail.commodities[].customsValue.amount` should be present as a two-decimal amount, not a long floating-point artifact.
7. If a commercial invoice or customs document is generated, confirm the displayed customs value is consistent with the rounded request value.

Expected Behaviour

What support should observe	How to confirm
FedEx label generation succeeds for the previously failing decimal-value case	Run the same or equivalent international shipment

What support should observe	How to confirm
FedEx does not reject the request because of malformed customs value precision	Review the failed-vs-fixed request result
The outbound customs amount is rounded to two decimals	Inspect <code>customsClearanceDetail.commodities[].customsValue.amount</code> in the request log
Merchant-entered customs intent is preserved	Compare the source value and the rounded request value

Merchant-Safe Explanation

You can explain this as a formatting correction for FedEx customs amounts. The app now sends a clean shipping value format that FedEx accepts more reliably, without asking the merchant to re-enter product data differently.

Common Questions & Troubleshooting

Q: The merchant says the label still fails. What should support check first?

Confirm the order is actually going through FedEx REST, then inspect the outbound customs amount in the request log and capture the exact FedEx error response.

Q: Does this change alter the merchant's actual customs valuation policy?

No. It is meant to normalize the number format sent to FedEx, not change the merchant's business logic.

Q: What if the request still shows too many decimals?

Capture the request log, source product/customs values, and any discount context, then escalate because the rounding fix is not applying to that path.

Support Escalation Packet

- › Story card: ZI-560
- › Ticket: #392156
- › Store URL, account UUID, order number, destination country, and timestamp
- › Screenshot of source product/customs value
- › Screenshot of FedEx REST request log showing `customsClearanceDetail.commodities[].customsValue.amount`
- › Screenshot of the FedEx response or label failure if the issue persists

Internal - Order Grid filter search should ignore leading and trailing spaces

Feature Summary

This change makes Order Grid filtering more forgiving by trimming leading and trailing spaces from the search input before matching records. The main support outcome is that pasted values such as order IDs, names, or tracking references should still match even if the input accidentally includes spaces at the beginning or end.

This is a UI and search-normalization fix. It should reduce false "no results" situations without changing the stored order data.

Toggles & Prerequisites

Check	What to confirm
Release	Confirm the store is on the build containing the filter-trim fix
Area	ORDERS tab / Order Grid filter input
Search sample	Use a value that is known to return a result without spaces
Toggle	No toggle detected locally

Step-by-Step Support Walkthrough

1. Open the PH Multi Carrier Shipping Label App and go to the ORDERS tab.
2. Choose a known value that already exists in the grid, such as Order Id, Customer name, Carrier name, Tags, or another supported filter target.
3. Run the filter once using the clean value and confirm the expected record appears.
4. Clear the filter and run the same search again with leading and trailing spaces added, for example .
5. Confirm the same result set appears and the search does not silently fail because of the extra spaces.
6. If the grid supports Add filter style matching, repeat the check on the most common support path, especially Order Id filtering.

Expected Behaviour

What support should observe	How to confirm
Search works with the clean value	Use a known order or reference
Search returns the same result when padded with spaces	Compare clean-input and padded-input results
No false empty-state appears only because of copied spaces	Repeat with a pasted value
Stored order data remains unchanged	Reopen the order record after the search

Merchant-Safe Explanation

You can explain this as a search usability improvement. If a merchant copies a value with extra spaces, the Order Grid should still find the right order instead of acting like nothing matches.

Common Questions & Troubleshooting

Q: Does this mean partial search rules changed?

No. This fix is about trimming accidental leading or trailing spaces, not changing the broader match rules.

Q: What if the order still does not appear?

Check whether the exact value exists, whether the wrong filter type is selected, and whether the order is outside the current view or status filter.

Support Escalation Packet

- › Story card: Internal Order Grid filter trim fix
- › Affected store URL and user
- › Exact value searched with and without spaces
- › Screenshot of the filter input and returned grid results

ZI-555 - FedEx REST postalCode null fix for HK / Taiwan

Feature Summary

This change addresses FedEx REST request failures where the postal-code field was being sent as null for Hong Kong or Taiwan scenarios. The intended result is that MCSL should now map and send an acceptable postal-code value for the affected address path instead of sending a JSON null that causes rate or label rejection.

Support should handle this as an address-payload fix for edge-region addressing. The merchant-facing outcome is that eligible Hong Kong and Taiwan shipments should rate or label normally without requiring workarounds in the order data.

Toggles & Prerequisites

Check	What to confirm
Release	Confirm the store is on the build containing ZI-555
Carrier account	FedEx REST is active
Scenario	Use an HK or Taiwan order or test shipment matching the reported Cybershaft flow
Address data	Confirm the order has the address data that should be mapped into the FedEx request
Toggle	No merchant-facing toggle detected locally

Step-by-Step Support Walkthrough

1. Confirm the affected shipment is going through FedEx REST and the destination or related address path matches the HK or Taiwan scenario covered by the card.
2. Open the source order and review the address values present in the store before testing.
3. Trigger a rate or label request in MCSL for the same shipment.
4. Confirm the request is no longer rejected for a null or invalid postal-code field.
5. Open the matching FedEx REST request log and inspect the address block used for the affected party in the request.
6. Request/log fields to verify: the relevant FedEx REST `address.postalCode` field should be sent as a usable string value and should not be sent as JSON `null`.
7. If the issue still occurs, capture the exact address block from the request and the FedEx response message before escalating.

Expected Behaviour

What support should observe	How to confirm
HK and Taiwan FedEx REST requests do not fail because of a null postal code	Re-run the previously failing scenario
The outbound address payload contains a real postal-code value or the platform's accepted mapped value	Inspect the request log

What support should observe	How to confirm
The merchant does not need a manual workaround such as editing a hidden payload field	Confirm the normal order flow works

Merchant-Safe Explanation

You can explain this as a regional address-format handling fix for FedEx REST. The app now sends the address details in a way that better matches what FedEx expects for the affected HK and Taiwan flow.

Common Questions & Troubleshooting

Q: The order still fails. What should support compare?

Compare the source order address, the FedEx request `address.postalCode` value, and the exact FedEx error response.

Q: Should support ask the merchant to invent a postcode manually?

Not as the first step. First confirm what the store already holds and whether the fix is present in the build.

Support Escalation Packet

- › Story card: ZI-555
- › Ticket: #391078
- › Store URL, order number, destination country/region, and timestamp
- › Source order address screenshot
- › FedEx REST request log screenshot showing the relevant `address.postalCode`
- › FedEx response screenshot if the request still fails

ZI-520 - Amazon Shipping India Verified Address (VA) API integration

Feature Summary

This card adds Amazon Shipping India verified-address support into the MCSL flow so address validation can happen through the Amazon Shipping India path instead of relying only on downstream shipment failure. The intended support outcome is better early detection of address issues and smoother Amazon Shipping India rate or label execution for eligible stores.

This should be treated as an Amazon Shipping India capability integration, not just a UI tweak. Support should verify both the account setup and the address-verification behavior. A new visual indicator is part of the change: a green tick mark means the address is validated, and a red cross mark means the address is not validated.

Toggles & Prerequisites

Check	What to confirm
Release	Confirm the store is on the build containing ZI-520
Carrier account	Amazon Shipping is connected in MCSL
Country scope	India flow / Amazon Shipping India account
Address scope	Use an address that should be accepted, and if possible one that should surface a validation issue
Pickup setup	Any Amazon Shipping India pickup-window or account prerequisites required for the tested shipment are already complete
Toggle	<code>accountUUID.address.deliverability.enabled: true</code> must be enabled for the target account

Step-by-Step Support Walkthrough

1. Open hamburger menu > Carriers and confirm Amazon Shipping is connected for the store.
2. Confirm the account is the India flow and the shipper/origin details are valid for Amazon Shipping India usage.
3. Use a test order with an India shipping address and trigger the normal Amazon Shipping rate or shipment flow.
4. Confirm the target account has `accountUUID.address.deliverability.enabled` set to `true` before validating the UI behavior.
5. Confirm the app does not silently bypass address verification for the tested scenario.
6. Check the new address-validation indicator in the relevant order or address flow:
 - › Green tick mark: Validated Address
 - › Red cross mark: Address is not validated
7. If the address is valid, confirm the flow proceeds normally into rate or shipment handling.
8. If you intentionally test an invalid or borderline address, confirm support gets a clear validation signal instead of an unexplained downstream failure.

9. Verify the Deliverability filter is available and can be used to filter the relevant orders or records by address-validation state.
10. Inspect the Amazon Shipping India logs for the address-verification step and capture the verified-address request/response block used for the run.

Expected Behaviour

What support should observe	How to confirm
Amazon Shipping India account can participate in the verified-address flow	Confirm request/log evidence from the tested run
A green tick mark appears for a validated address	Check the new status icon in the tested flow
A red cross mark appears when the address is not validated	Use an invalid or non-validated address scenario
Deliverability filter is available for support use	Confirm the filter is visible and usable in the relevant grid or listing
Valid addresses move forward without avoidable address rejection	Run a valid India shipment
Invalid or incomplete addresses surface a clear validation signal	Run a controlled invalid-address check if available
Support can capture address-verification evidence from logs	Open the Amazon Shipping India request/response logs

Merchant-Safe Explanation

You can explain this as stronger address validation for Amazon Shipping India. The app can now check address acceptability earlier so merchants get fewer avoidable shipment failures later in the process, and the support team can also see a clearer validated/not-validated status in the app.

Common Questions & Troubleshooting

Q: Which log should support inspect if the shipment fails?

Inspect the Amazon Shipping India request/response logs for the verified-address or address-validation step before looking only at the final shipment failure.

Q: Does this replace normal carrier-account setup requirements?

No. The carrier account, origin, and any Amazon Shipping India prerequisites still need to be correct.

Q: What does the new icon mean?

A green tick mark means the address is validated. A red cross mark means the address is not validated.

Q: What should support verify if the icon or Deliverability filter is missing?

First confirm the account-level toggle `accountUUID.address.deliverability.enabled` is enabled for the target account and that the store is on the correct build.

Support Escalation Packet

› Story card: ZI-520

- › Ticket: #389826
- › Store URL, Amazon Shipping account reference, test order number, and timestamp
- › Toggle state for `accountUUID.address.deliverability.enabled`
- › Source address screenshot
- › Screenshot showing green tick or red cross status
- › Screenshot showing the Deliverability filter
- › Amazon Shipping India log screenshot covering the address-verification step
- › Final shipment or rate response screenshot if the flow still fails

ZI-557 - Shipping zone postcodes should drive checkout rates on WooCommerce Shipping Services

Feature Summary

This change addresses silent failures where configured shipping-zone postcodes were not actually influencing checkout rates in WooCommerce Shipping Services. The intended result is that postcode-targeted zones now participate correctly in rate selection, so the right checkout rates appear for matching postcode scenarios instead of failing quietly.

Support should treat this as a configuration-to-checkout mapping fix. The merchant-facing outcome is that a postcode-based shipping zone behaves as configured at checkout.

Toggles & Prerequisites

Check	What to confirm
Release	Confirm the WooCommerce store is on the build containing ZI-557
Platform	WooCommerce Shipping Services
Zone setup	A shipping zone exists with one or more configured postcodes
Test data	Use one address that should match the configured postcode list and one that should not
Toggle	No toggle detected locally

Step-by-Step Support Walkthrough

1. Open the WooCommerce admin and review the relevant shipping-zone configuration used by the merchant.
2. Confirm the zone includes the target postcode entries and that the values were saved correctly.
3. Run a checkout test using a destination postcode that should match the configured zone.
4. Confirm the expected rate appears at checkout and is driven by the matching postcode rule.
5. Run a second checkout test using a postcode that should not match and confirm the result differs appropriately.
6. If the rate does not appear, inspect the available rate log, request log, or rule-matching evidence for the checkout run and capture the postcode used in the test.

Expected Behaviour

What support should observe	How to confirm
Matching postcodes influence checkout rate selection	Run checkout with a matching postcode
Non-matching postcodes do not accidentally receive the same zone rate	Run checkout with a control postcode
Failure is no longer silent for the reported path	Compare before/after or expected-vs-actual outcome

What support should observe	How to confirm
Saved zone postcode configuration is honored by the checkout engine	Review zone setup and checkout result together

Merchant-Safe Explanation

You can explain this as a WooCommerce checkout rate fix for postcode-based shipping zones. If the merchant configures a zone for specific postcodes, the checkout should now respect that rule more reliably.

Common Questions & Troubleshooting

Q: What should support verify first when the rate still does not show?

Check the exact postcode list saved in the zone, the destination postcode used in the test, and whether another zone or method is taking precedence.

Q: Are wildcard or range formats automatically supported?

Do not assume so from this card. Validate using the exact postcode-entry style already supported by the merchant's configuration flow.

Support Escalation Packet

- › Story card: ZI-557
- › Ticket: #384856
- › WooCommerce store URL, zone name, configured postcode list, and test postcode
- › Screenshot of zone configuration
- › Screenshot of checkout result for matching and non-matching test cases

Internal - FedEx REST rates are failing for GU and VI

Feature Summary

This change targets FedEx REST rate failures for Guam (GU) and the U.S. Virgin Islands (VI). Support should treat this as a region-handling and address-qualification fix so these destination scenarios no longer fail during rating when the rest of the shipment data is valid.

The merchant-facing result should be that GU and VI orders can request rates normally instead of looking like unsupported or malformed destinations when they should be serviceable.

Toggles & Prerequisites

Check	What to confirm
Release	Confirm the store is on the build containing the GU / VI rate fix
Carrier account	FedEx REST is active
Scenario	Use a GU and/or VI destination that previously failed
Address data	Confirm state/region, country code, postal code, and destination details are populated correctly in the order
Toggle	No merchant-facing toggle detected locally

Step-by-Step Support Walkthrough

1. Confirm the shipment is using FedEx REST and that the destination is Guam or the U.S. Virgin Islands.
2. Open the order and verify the destination address fields before running the test.
3. Trigger a live FedEx REST rate request for the order.
4. Confirm rates return instead of failing at destination validation.
5. Inspect the FedEx REST request log for the tested run.
6. Request/log fields to verify: `destination.address.countryCode`, `address.stateOrProvinceCode`, and `address.postalCode` should be populated consistently for the GU or VI scenario used in the test.
7. If the flow still fails, capture the exact FedEx error message and the destination address block from the request log.

Expected Behaviour

What support should observe	How to confirm
FedEx REST returns rates for valid GU and VI scenarios	Run the target rate request
The destination is not rejected because of territory-handling mistakes	Review the FedEx response
Request address fields are populated consistently	Inspect the request log

Merchant-Safe Explanation

You can explain this as a FedEx destination-handling correction for specific U.S. territory scenarios. The app should now pass those addresses through the FedEx REST rating flow more reliably.

Common Questions & Troubleshooting

Q: The merchant says GU still fails. What should support capture?

Capture the order address, request address block, FedEx response message, and whether the same account can rate nearby control scenarios.

Q: Is this only a label fix?

No. The title specifically points to the rates path, so support should test rating first.

Support Escalation Packet

- › Story card: Internal GU / VI FedEx REST rate fix
- › Store URL, FedEx account, destination territory, postal code, and timestamp
- › Source order address screenshot
- › FedEx REST request log screenshot for destination address fields
- › FedEx response screenshot if the rate request still fails

Internal - Store-based config / toggle for enabling mock rates for FedEx app and MCSL

Feature Summary

This item introduces a store-based internal toggle or configuration path that allows QA/support to enable mock FedEx rates for a specific store in the FedEx app and MCSL when live FedEx rates are unavailable. This is meant for internal QA continuity and controlled troubleshooting, not as a normal merchant-facing shipping setup.

Support should use this only when instructed by QA, development, or a documented internal testing process. The key behavior is store-scoped enablement: one store can be switched to mock-rate handling without implying a global carrier change for everyone else.

Toggles & Prerequisites

Check	What to confirm
Usage scope	Internal QA / support usage only
Store identity	Exact store URL, account UUID, and if needed store UUID are known
Toggle path	Confirm the current live config source or toggle service used by the team
Carrier scope	FedEx app and MCSL mock-rate behavior for the targeted store only
Safety check	Confirm live merchant expectations before enabling anything that changes displayed rates

Step-by-Step Support Walkthrough

1. Confirm QA or development has approved the use of mock rates for the target store.
2. Capture the exact store identifier and verify you are not applying the change globally.
3. Apply or confirm the store-based mock-rate config in the team's live toggle/config system.
4. Reopen the store flow and trigger the FedEx rate request in the FedEx app or MCSL path covered by the card.
5. Confirm a rate response is now available through the mock path for the targeted store.
6. Clearly document in the support ticket or QA notes that mock rates were enabled, including who approved it and when.
7. When testing is complete, confirm whether the toggle should remain on for that store or be removed.

Expected Behaviour

What support should observe	How to confirm
Mock-rate behavior can be enabled for one store without implying a platform-wide rollout	Verify the config is store-scoped
QA can continue testing when live FedEx rates are unavailable	Trigger a rate flow after enablement

What support should observe	How to confirm
The change is traceable and reversible	Record approver, timestamp, and current toggle state

Merchant-Safe Explanation

Do not describe this as a normal merchant feature unless product explicitly approves that positioning. Internally, it is a store-specific testing fallback used when live FedEx rates are unavailable and QA still needs to continue a controlled validation flow.

Common Questions & Troubleshooting

Q: Should support enable mock rates for any merchant reporting a rate issue?

No. This is an internal controlled path. Confirm approval and store scope first.

Q: What if rates still do not appear after the toggle is enabled?

Capture the store identifiers, toggle state, exact app path tested, and the post-toggle rate/request log evidence before escalating.

Support Escalation Packet

- › Story card: Internal store-based mock-rates toggle
- › Store URL, account UUID, store UUID if available
- › Name of approver and timestamp
- › Screenshot or exported proof of current toggle/config state
- › Screenshot of the rate flow after mock-rate enablement